

METODOLOGÍA I

Trabajo Práctico — El ciclo completo de roles (y su límite)

OBJETIVO GENERAL

Aplicar los patrones de prompting de la Semana 2 (anatomía del prompt, few-shot, role prompting, cadena de razonamiento) anclados en los roles concretos del ciclo de desarrollo de software — Programador, Revisor, Tester, Arquitecto — sumando refactorización, debugging asistido y documentación. Como cierre, experimentar el límite de todo lo anterior mediante un pedido sin ninguna especificación previa, para reconocer que la variabilidad resultante no es un problema de redacción del prompt sino la ausencia de un objetivo verificable.

MARCO TEÓRICO

Rol del ciclo de desarrollo	Patrón de prompting que ancla
Programador	Anatomía del prompt aplicada a código nuevo: firma exacta, comportamiento ante entradas inválidas, estilo del proyecto.
Revisor	Restricción de alcance: un único criterio de revisión a la vez (legibilidad, manejo de errores, seguridad), no un pedido genérico.
Tester	Exigencia explícita de casos válidos, inválidos y límite — no aceptar solo el camino feliz que ofrece el modelo por defecto.
Debugging asistido	Cadena de razonamiento (Chain of Thought): pedir la causa raíz antes de proponer cualquier corrección.
Refactorización asistida	Restricciones explícitas de alcance (comportamiento externo idéntico, no cambiar firma) + verificación con tests antes y después.
Arquitecto	Role Prompting aplicado a decisiones estructurales: pedir alternativas con trade-offs, no una única solución.
Documentación	Especificar público destinatario y nivel de detalle, evitando comentarios que repiten el nombre de la función.

PARTE A — CICLO COMPLETO DE ROLES

Elegí una función o clase propia (de un TP anterior, un proyecto personal, o un ejercicio del cuadernillo de la cátedra) y documentá 7 fichas, una por cada rol. Cada ficha debe incluir tres partes: el **prompt completo** que

usaste, la **respuesta relevante** del modelo (resumida si es larga), y una **observación** tuya sobre qué patrón de la unidad aplica y por qué.

1. Rol Programador

Patrón que aplica: anatomía del prompt aplicada a código nuevo (Semana 2). Elegí una función o método nuevo relacionado con tu caso y escribí un único prompt que deje explícitos los tres elementos siguientes — no alcanza con pedir "hacé una función que haga X":

- **Firma exacta:** nombre, parámetros con su tipo, y qué devuelve.
- **Comportamiento ante entradas inválidas:** qué debe pasar en al menos un caso concreto (excepción puntual, valor de retorno, o mensaje de error).
- **Estilo del proyecto:** convención de nombres, uso de type hints/docstrings, y cualquier patrón que ya uses en tu código real — no un estilo genérico que invente el modelo.

2. Rol Revisor

Patrón que aplica: restricción de alcance, un único criterio de revisión a la vez. Pedile al modelo que revise el código de la Ficha 1, pero elegí en tu prompt UN SOLO criterio (legibilidad, manejo de errores o seguridad) — nunca "revisá esto" sin más. Dejá explícito en el prompt que el modelo se limite a ese criterio y no opine sobre los demás.

3. Rol Tester

Patrón que aplica: exigencia explícita de cobertura, no aceptar solo el camino feliz. Pedile al modelo una batería de tests para la función de la Ficha 1, nombrando en el prompt los tres tipos de caso que tiene que cubrir — si el prompt dice solo "generá tests", el modelo tiende a devolverte únicamente el camino feliz:

- Al menos un caso de entrada típica/válida.
- Al menos un caso de entrada inválida.
- Al menos un caso límite (borde).

4. Debugging con cadena de razonamiento

Patrón que aplica: Chain of Thought. Si el Tester (Ficha 3) encontró un bug, o inventó uno plausible, describí el síntoma en el prompt (qué se esperaba vs. qué pasó de verdad) y pedí explícitamente que el modelo explique la causa raíz ANTES de proponer cualquier corrección — no un pedido directo de "arreglá esto". Mostrá los dos pasos por separado: primero el razonamiento, después la corrección.

5. Refactorización asistida

Patrón que aplica: restricciones explícitas de alcance + verificación. Elegí una parte del código de la Ficha 1 para refactorizar y fijá en el prompt qué tiene que mantenerse idéntico (firma, nombre, tipo de retorno) y qué queda fuera de alcance (por ejemplo, "no agregues funcionalidad nueva"). Después de aplicar el cambio, corré los tests de la Ficha 3 antes y después del refactor y documentá el resultado (pasaron / no pasaron) — esta verificación es parte obligatoria de la ficha, no un paso opcional.

6. Rol Arquitecto

Patrón que aplica: Role Prompting aplicado a decisiones estructurales. Planteá una decisión de diseño real de tu caso (por ejemplo, cómo separar dos responsabilidades, qué estructura de datos usar) y pedile al modelo, en el rol de arquitecto de software, 2-3 alternativas con sus ventajas y desventajas — nunca le pidas una única solución.

7. Documentación automática

Patrón que aplica: especificar público destinatario y nivel de detalle. Documentá la función final para dos públicos distintos, dejando el público explícito en cada prompt:

- Documentación técnica para un dev del equipo: contratos (tipos, excepciones) y casos de borde.
- README público: cómo usar la función, sin detalle interno de implementación.

En ambos casos, pedile explícitamente al modelo que evite comentarios que solo repiten el nombre de la función sin agregar información (por ejemplo, `# suma dos números` sobre una función llamada `sumar`).

PARTE B — EL EXPERIMENTO: PROMPTING SIN ESPECIFICACIÓN

Esta parte rompe a propósito todo lo que armaste en la Parte A.

- Elegí un pedido deliberadamente ambiguo, por ejemplo: *"Hacé un sistema de reservas para un restaurante."* (podés usar este mismo o proponer uno de alcance similar).
- Escribíselo al modelo tal cual, **sin** anatomía de prompt, sin rol, sin restricciones ni criterios de éxito — el opuesto exacto de las 7 fichas de la Parte A.
- Guardá la respuesta completa que obtuviste, sin editarla ni "arreglarla" después.

Reflexión final (mínimo 150 palabras): ¿qué decisiones tomó el modelo por su cuenta que en la Parte A vos habías especificado explícitamente (formato, casos de borde, alcance, criterios de éxito)? ¿Por qué el problema acá no es "un prompt mal escrito"? Cerrá conectando esto con la idea de un sistema que actúa en varios pasos sin un objetivo verificable.

CONSIDERACIONES

- ✅ **Elección del caso:** podés reusar código de un TP anterior, un proyecto personal o un ejercicio del cuadernillo — no hace falta escribir un proyecto nuevo desde cero.
- ✅ **Ficha completa:** una ficha con solo el prompt, sin la respuesta del modelo ni tu observación, se considera incompleta.
- ✅ **Parte B sin retoque:** la respuesta del experimento se documenta tal cual salió, incluso si el resultado es pobre o inesperado — eso es justamente lo que hay que analizar.
- ✅ **No hay "respuesta correcta" en la Parte B:** se evalúa la calidad de la reflexión, no el resultado del experimento en sí.

CONCLUSIONES ESPERADAS

- Aplicar con criterio propio los patrones de prompting de la Semana 2, anclados en un rol concreto del ciclo de desarrollo, en vez de usarlos de forma genérica.
- Distinguir cuándo un resultado inconsistente del modelo es un problema de prompting (falta de especificación) y cuándo es síntoma de algo más profundo: un sistema actuando en varios pasos sin un objetivo verificable.
- Llegar a la Semana 4 con la pregunta ya planteada en carne propia: ¿qué hace falta para que ese sistema autónomo sepa cuándo terminó bien?